

Fiche 317 — Erreurs, avertissements, débogage et profilage

Matière	Code · Langage R
Cours source	R Core Team, <i>R Language Definition</i> 4.6.1 — chapitre 8 « Exception handling » (§8.1 <code>stop</code> , §8.2 <code>warning</code> , §8.3 <code>on.exit</code> , §8.4 options d'erreur) et chapitre 9 « Debugging » (§9.1 <code>browser</code> , §9.2 <code>debug</code> / <code>undebug</code> , §9.3 <code>trace</code> / <code>untrace</code> , §9.4 <code>traceback</code>)
Sources d'appoint	R Core Team, <i>Writing R Extensions</i> 4.6.1 — chapitre 3 « Tidying and profiling R code » (§3.1 mise au propre, §3.2 profilage en vitesse)
Difficulté	Intermédiaire — peu de concepts, beaucoup de réflexes
Temps d'étude estimé	1 h 15
Prérequis	Fiches 309, 310, 316 (fonctions, environnements, appels)
Concepts clés	<code>stop()</code> , <code>warning()</code> , l'option <code>warn</code> et ses quatre régimes, <code>last.warning</code> , <code>warnings()</code> , <code>on.exit()</code> , options <code>error</code> et <code>warning.expression</code> , <code>browser()</code> , les cinq commandes du débogueur, <code>debug()</code> / <code>undebug()</code> , <code>trace()</code> / <code>untrace()</code> , <code>.Traceback</code> et <code>traceback()</code> , <code>keep.source</code> , <code>removeSource()</code> , <code>dump()</code> , <code>Rprof()</code> , <code>summaryRprof()</code>
À retenir en priorité	Les quatre valeurs de <code>warn</code> · <code>on.exit</code> est exécuté même sur avertissement · les cinq commandes <code>RET c n where Q</code> · <code>traceback()</code> après une erreur · profiler avant d'optimiser.

Vue d'ensemble

DEUX MECANISMES « Functions such as `stop` or `warning` can be CALLED DIRECTLY or OPTIONS such as "`warn`" can be used to control the handling of problems. »

`stop()` « HALTS the evaluation of the current expression, PRINTS the message argument and RETURNS EXECUTION TO TOP-LEVEL »

`warning()` son comportement depend de l'option "`warn`" :
`warn < 0` les avertissements sont IGNORES

```
warn = 0   STOCKES, imprimes APRES la fonction de haut niveau (default)
warn = 1   imprimes AU MOMENT ou ils surviennent
warn >= 2  TRANSFORMES EN ERREURS
a warn = 0 : moins de 10 -> imprimes ; plus de 10 -> un simple compte
              last.warning garde le vecteur ; warnings() y accede
```

```
on.exit()  « the effect ... is to STORE THE VALUE OF THE BODY so that it will
            be EXECUTED WHEN THE FUNCTION EXITS »
            « guaranteed to be executed ... EITHER DIRECTLY OR AS THE RESULT
            OF A WARNING »
```

```
TROIS OPTIONS  warn · warning.expression · error
                « Expressions installed by options("error") are evaluated
                BEFORE calls to on.exit are carried out. »
```

```
DEBOGUEUR      browser() s'arrete ICI          debug(f) s'arrete a CHAQUE appel
                trace(f)  imprime l'appel       traceback() la pile APRES l'erreur
```

CINQ COMMANDES DU DEBOGUEUR

```
RET    instruction suivante (debug) / continuer (browser)
c cont  continuer
n       instruction suivante -- MARCHE AUSSI depuis le browser
where   afficher la PILE D'APPELS
Q       arreter et remonter au sommet IMMEDIATEMENT
```

```
PROFILER      Rprof("f.out") ... Rprof(NULL) puis summaryRprof()
                echantillonne toutes les 20 ms QUELLE LIGNE DE QUELLE FONCTION
```

Le problème posé. « *The exception handling facilities in R are provided through two mechanisms. Functions such as `stop` or `warning` can be called directly or options such as `"warn"` can be used to control the handling of problems.* » (§8)

INTUITION — SIGNALER ET RÉGLER SONT DEUX CHOSSES DISTINCTES.

Le code **signale** (`stop` , `warning`) ; l'utilisateur **décide** ce qui en est fait (`options`). Un même `warning()` peut être ignoré, différé, immédiat ou fatal — **sans que le code change**. C'est ce partage qui permet à `options(warn = 2)` de transformer un script indulgent en script strict, le temps d'un diagnostic.

● Concept 1 — Signaler une erreur

DÉFINITION (§8.1).

« *A call to `stop` halts the evaluation of the current expression, prints the message argument and returns execution to top-level.* »

Trois effets en une phrase : arrêter, afficher, revenir au sommet. Rien de ce qui suit dans la fonction n'est exécuté — c'est ce qui rend le garde-fou du compte en banque efficace (fiche 310) :

```
withdraw = function(amount) {  
  if (amount > total)  
    stop("You don't have that much money!\n")  
  total <- total - amount      # <- jamais atteinte si le stop se declenche  
  ...  
}
```

● Concept 2 — Avertir, et les quatre régimes de `warn`

DÉFINITION (§8.2).

« *The function `warning` takes a single argument that is a character string. The behaviour of a call to `warning` depends on the value of the option `"warn"`.* »

Valeur de <code>warn</code>	Comportement
négative	« <i>warnings are ignored</i> »
zéro (le défaut)	« <i>they are stored and printed after the top-level function has completed</i> »
un	« <i>they are printed as they occur</i> »
2 ou plus	« <i>warnings are turned into errors</i> »

Le détail du régime par défaut (§8.2). « If `"warn"` is zero (the default), a variable `Last.warning` is created and the messages associated with each call to `warning` are stored, sequentially, in this vector. If there are fewer than 10 warnings they are printed after the function has finished evaluating. If there are more than 10 then a message indicating how many warnings occurred is printed. In either case `Last.warning` contains the vector of messages, and `warnings` provides a way to access and print it. »

MÉTHODE — LES DEUX RÉGLAGES QUI SERVENT EN PRATIQUE.

- `options(warn = 1)` quand on cherche où un avertissement se produit : à `warn = 0` il est affiché **après** la fonction de haut niveau, donc **détaché de sa cause**. À `warn = 1` il apparaît **au bon endroit** dans la trace.
- `options(warn = 2)` quand on veut **traiter un avertissement comme un bug** : il devient une erreur, donc `traceback()` (concept 5) donne **la pile d'appels complète**.

Et **le seuil de dix** explique le « There were 12 warnings (use `warnings()` to see them) » que tout utilisateur a rencontré : au-delà de dix, R **ne les imprime plus**, il les **compte**. `warnings()` les récupère.

● Concept 3 — `on.exit()`

DÉFINITION (§8.3).

« A function can insert a call to `on.exit` at any point in the body of a function. The effect of a call to `on.exit` is to store the value of the body so that it will be executed when the function exits. This allows the function to change some system parameters and to ensure that they are reset to appropriate values when the function is finished. »

« The `on.exit` is guaranteed to be executed when the function exits either directly or as the result of a warning. »

« An error in the evaluation of the `on.exit` code causes an immediate jump to top-level without further processing of the `on.exit` code. »

« `on.exit` takes a single argument which is an expression to be evaluated when the function is exited. »

C'est la garantie qui manquait à l'idiome `oldpar` de la fiche 314. Le cours de graphiques écrit :

```
oldpar <- par(no.readonly = TRUE)
# ... commandes de trace ...
par(oldpar)
```

Si une commande intermédiaire échoue, la dernière ligne n'est jamais atteinte — et les réglages de l'utilisateur restent modifiés. `on.exit` supprime ce risque :

```
oldpar <- par(no.readonly = TRUE)
on.exit(par(oldpar))
# ... commandes de trace ...
```

Le même patron vaut pour tout ce qui doit être rendu : un périphérique graphique ouvert (`dev.off()` , fiche 314), un `sink()` ouvert (fiche 300), un fichier temporaire, un répertoire de travail changé.

● Concept 4 — Les options d'erreur

DÉFINITION (§8.4).

« *There are a number of options variables that can be used to control how R handles errors and warnings.* »

Option	Rôle
<code>warn</code>	« <i>Controls the printing of warnings.</i> » — concept 2
<code>warning.expression</code>	« <i>Sets an expression that is to be evaluated when a warning occurs. The normal printing of warnings is suppressed if this option is set.</i> »
<code>error</code>	« <i>Installs an expression that will be evaluated when an error occurs. The normal printing of error messages and warning messages precedes the evaluation of the expression.</i> »

L'ordre d'exécution (§8.4). « *Expressions installed by `options("error")` are evaluated before calls to `on.exit` are carried out.* »

Cela compte : au moment où l'expression d'erreur s'exécute, **les nettoyages n'ont pas encore eu lieu**. C'est précisément ce qui rend un `options(error = recover)` utile — l'état des cadres d'appel est **encore intact**.

L'exemple du manuel (§8.4). « *One can use `options(error = expression(q("yes")))` to get R to quit when an error has been signalled. In this case an error will cause R to shut down and the global environment will be saved.* »

Usage typique : le script par lots. En `R CMD BATCH` (fiche 300), un script qui continue après une erreur produit des résultats faux. Cette option le fait **s'arrêter net** — en sauvegardant l'espace de travail pour l'autopsie.

● Concept 5 — Les outils de débogage

Cadrage (§9). « *Debugging code has always been a bit of an art. R provides several tools that help users find problems in their code. These tools halt execution at particular points in the code and the current state of the computation can be inspected.* »

« *Most debugging takes place either through calls to `browser` or `debug`. Both of these functions rely on the same internal mechanism and both provide the user with a special prompt. Any command can be typed at the prompt. The evaluation environment for the command is the currently active environment. This allows you to examine the current state of any variables.* »

5.1 Les cinq commandes spéciales

Règle (§9). « *There are five special commands that R interprets differently.* »

Commande	Effet
<code>RET</code>	« <i>Go to the next statement if the function is being debugged. Continue execution if the browser was invoked.</i> »
<code>c</code> / <code>cont</code>	« <i>Continue the execution.</i> »
<code>n</code>	« <i>Execute the next statement in the function. This works from the browser as well.</i> »
<code>where</code>	« <i>Show the call stack</i> »
<code>Q</code>	« <i>Halt execution and jump to the top-level immediately.</i> »

Le collision de noms (§9). « *If there is a local variable with the same name as one of the special commands listed above then its value can be accessed by using `get`. A call to `get` with the name in quotes will retrieve the value in the current environment.* »

Une variable locale nommée `c` ou `n` **est inaccessible directement** dans le débogueur — la lettre est interprétée comme une commande. `get("c")` la récupère. C'est exactement l'exemple donné au §9.1.

La limite (§9). « *The debugger provides access only to interpreted expressions. If a function calls a foreign language (such as C) then no access to the statements in that language is provided. Execution will halt on the next statement that is evaluated in R. A symbolic debugger such as `gdb` can be used to debug compiled code.* » — voir fiche 320.

5.2 `browser()` — s'arrêter ici

DÉFINITION (§9.1).

« *A call to the function `browser` causes R to halt execution at that point and to provide the user with a special prompt. Arguments to `browser` are ignored.* »

```

> foo <- function(s) {
+   c <- 3
+   browser()
+ }
> foo(4)
Called from: foo(4)
Browse[1]> s
[1] 4
Browse[1]> get("c")
[1] 3

```

L'exemple est choisi pour montrer les deux choses : on inspecte `s`, l'argument, **dans son environnement d'évaluation** ; et `c`, variable locale, exige `get("c")` parce que `c` est une commande du débogueur.

5.3 `debug()` — s'arrêter à chaque appel

DÉFINITION (§9.2).

« *The debugger can be invoked on any function by using the command `debug(fun)`. Subsequently, each time that function is evaluated the debugger is invoked. The debugger allows you to control the evaluation of the statements in the body of the function. Before each statement is executed the statement is printed out and a special prompt provided.* »

« *Debugging is turned off by a call to `undebug` with the function as an argument.* »

```

> debug(mean.default)
> mean(1:10)
debugging in: mean.default(1:10)
debug: {
  if (na.rm) x <- x[!is.na(x)]
  trim <- trim[1]
  n <- length(c(x, recursive = TRUE))
  ...
}
Browse[1]>
debug: if (na.rm) x <- x[!is.na(x)]

```



```
Browse[1]>
debug: trim <- trim[1]
Browse[1]>
debug: n <- length(c(x, recursive = TRUE))
Browse[1]> c
exiting from: mean.default(1:10)
[1] 5.5
```

CONTRÔLE — TROIS CHOSES À REMARQUER DANS CETTE TRACE.

1. `debug(mean.default)` , pas `debug(mean)` : `mean` est **une générique** (fiche 311), son corps se résume à `UseMethod("mean")` . C'est **la méthode** qu'il faut instrumenter.
2. **Le corps entier s'affiche d'abord**, puis **une instruction à la fois** ; chaque `RET` avance d'une ligne.
3. `c` **termine le pas-à-pas** et laisse la fonction s'achever — d'où le `[1] 5.5` final.

5.4 `trace()` — voir les appels

DÉFINITION (§9.3).

« *Another way of monitoring the behaviour of R is through the trace mechanism. `trace` is called with a single argument that is the name of the function you want to trace. **The name does not need to be quoted but for some functions you will need to quote the name in order to avoid a syntax error.*** »

« *When `trace` has been invoked on a function then every time that function is evaluated the call to it is printed out. This mechanism is removed by calling `untrace` .* »

```
> trace("[<-")
> x <- 1:10
> x[3] <- 4
trace: "[<->(*tmp*, 3, value = 4)
```

Cet exemple vaut une démonstration. Il rend **visible** le mécanisme ``*tmp*`` de la fiche 302 : `x[3] <- 4` est **réellement** l'appel ``[<-`(*tmp*, 3, value = 4)` . Ce que le manuel décrivait comme « *as if the following had been executed* » est ici **observé**.

Et il illustre la réserve sur les guillemets : `trace([<-)` serait **une erreur de syntaxe** — `[<-` n'est pas un identifiant (fiche 300).

La différence avec `debug` : `trace` **n'interrompt pas**. Il affiche l'appel et laisse continuer — utile quand on veut savoir **combien de fois** et **avec quels arguments** une fonction est appelée, sans arrêter le flux.

5.5 `traceback()` — la pile après coup

DÉFINITION (§9.4).

« *When an error has caused a jump to top-level a special variable called `.Traceback` is placed into the base environment. `.Traceback` is a character vector with one entry for each function call that was active at the time the error occurred. An examination of `.Traceback` can be carried out by a call to `traceback`.* »

C'est le premier réflexe après une erreur inattendue. Le message d'erreur dit **quoi** ; `traceback()` dit **où**, en remontant toute la chaîne d'appels.

MÉTHODE — POURQUOI CELA RÉSOUT LES ERREURS « INCOMPRÉHENSIBLES ».

Les fiches précédentes ont montré plusieurs pannes **différées** : une classe posée sans vérification qui échoue au premier générique (fiche 311), un `drop` oublié qui casse `colMeans` (fiche 302), un `predict()` aux noms de variables discordants (fiche 313). Dans tous ces cas, **le message ne parle pas de la cause** — mais `traceback()` montre **la chaîne** qui y mène.

● Concept 6 — Mettre au propre

Cadrage (Writing R Extensions §3). « *R code which is worth preserving in a package and perhaps making available for others to use is worth documenting, tidying up and perhaps optimizing.* »

Règle (§3.1). « *R treats function code loaded from packages and code entered by users differently. By default code entered by users has the source code stored internally, and when the function is listed, the original source is reproduced. Loading code from a package (by default) discards the source code, and the function listing is re-created from the parse tree of the function.* »

« *Normally keeping the source code is a good idea, and in particular it avoids comments being removed. However, we can make use of the ability to re-create a function listing from its parse tree to produce a tidy version of the function, for example with consistent indentation and spaces around operators.* »

MÉTHODE — LES DEUX FAÇONS DE RENONCER À LA SOURCE (§3.1).

1. « *The option `keep.source` can be set to `FALSE` before the code is loaded into R.* »
2. « *The stored source code can be removed by calling the `removeSource()` function* » : `myfun`
`<- removeSource(myfun)` .

La recette complète (§3.1) — pour mettre au propre un fichier `myfuns.R` , écrire `tidy.R` :

```
source("myfuns.R", keep.source = FALSE)
dump(ls(all.names = TRUE), file = "new.myfuns.R")
```

« and run R with this as the source file, for example by `R --vanilla < tidy.R` or by pasting into an R session. **Then the file `new.myfuns.R` will contain the functions in alphabetical order in the standard layout. Warning : comments in your functions will be lost.** »

CONTRÔLE — DÉCODER CES DEUX LIGNES.

`keep.source = FALSE` fait **oublier le texte d'origine** ; les fonctions n'existent plus que comme **arbres syntaxiques** (fiche 316). `ls(all.names = TRUE)` liste **tous** les objets, **y compris ceux dont le nom commence par un point** (fiche 300). `dump()` les réécrit **à partir de l'arbre** — donc dans la **mise en page canonique**. Et `R --vanilla` (fiche 300) garantit qu'aucun fichier de démarrage n'interfère.

La recommandation de style (§3.1). « *The standard format provides a good starting point for further tidying. Although the deparsing cannot do so, we recommend the consistent use of the preferred assignment operator `<-` (rather than `=`) for assignment. Many package authors use a version of Emacs ... using the **ESS[S]** mode.* »

● Concept 7 — Profiler

DÉFINITION (WRITING R EXTENSIONS §3.2).

« *The command `Rprof` is used to control profiling. Profiling works by recording at fixed intervals (by default every 20 msecs) which line in which R function is being used, and recording the results in a file (default `Rprof.out` in the working directory). Then the function `summaryRprof` or the command-line utility `R CMD Rprof Rprof.out` can be used to summarize the activity.* »

(note 2 : « *For Unix-alikes by default these are intervals of CPU time, and for Windows of elapsed ("wall-clock") time.* »)

Le patron d'usage, tiré de l'exemple du manuel (d'après Venables & Ripley, 2002, pp. 225-6) :

```
Rprof("boot.out")
storm.boot <- boot(rs, storm.bf, R = 4999)  # slow enough to profile
Rprof(NULL)
```

puis

```
R CMD Rprof boot.out
```

La sortie donne deux tableaux, et le manuel en explique les colonnes :

« *Total seconds : time spent in function and callees. Self seconds : time spent in function alone.* »

Colonne	Ce qu'elle mesure
total	le temps passé dans la fonction et dans ce qu'elle appelle
self	le temps passé dans la fonction seule

La distinction est décisive. Une fonction peut avoir un `total` de 100 % et un `self` de 0,2 % — c'est le cas de `boot` dans l'exemple : elle **ne fait rien elle-même**, elle appelle. **On optimise ce qui a un `self` élevé.**

« *This often produces surprising results and can be used to identify bottlenecks or pieces of R code that could benefit from being replaced by compiled code.* »

Trois mises en garde du manuel (§3.2).

1. « *profiling does impose a small performance penalty* » ;
2. « *the output files can be very large if long runs are profiled at the default sampling interval* » ;
3. « *Profiling short runs can sometimes give misleading results. R from time to time performs garbage collection to reclaim unused memory, and this takes an appreciable amount of time which profiling will charge to whichever function happens to provoke it. It may be useful to compare profiling code immediately after a call to `gc()` with a profiling run without a preceding call to `gc`.* »

La troisième est la plus subtile : le ramasse-miettes est **imputé à la fonction qui l'a déclenché**, laquelle n'y est pour rien. Sur un profil court, cela peut désigner **un coupable innocent**.

« *More detailed analysis of the output can be achieved by the tools in the CRAN packages `proftools` and `profr` : in particular these allow call graphs to be studied.* »

Énoncé. Une fonction appelée dans une boucle plante « une fois sur vingt », avec un message qui ne mentionne pas son code. Quel enchaînement d'outils ?

Étape 1 — obtenir la pile. Après l'erreur, `traceback()` . « *.Traceback is a character vector with one entry for each function call that was active at the time the error occurred* » (§9.4). Elle nomme **la chaîne complète**, du haut niveau jusqu'à la fonction fautive — ce que le message d'erreur seul ne dit pas.

Étape 2 — si l'erreur n'est qu'un avertissement devenu fatal ailleurs. `options(warn = 2)` : « **if it is 2 (or larger) warnings are turned into errors** » (§8.2). L'avertissement devient une erreur, donc **traçable** par l'étape 1.

Étape 3 — si l'avertissement s'affiche loin de sa cause. `options(warn = 1)` : « **they are printed as they occur** ». À `warn = 0`, ils sont affichés **après la fonction de haut niveau** — donc détachés du contexte.

Étape 4 — entrer dans la fonction. `debug(mafonction)` : « **each time that function is evaluated the debugger is invoked ... before each statement is executed the statement is printed out** » (§9.2). On avance par `RET`, on inspecte les variables au prompt — « **the evaluation environment for the command is the currently active environment** ».

Étape 5 — sauter directement au point suspect. Plutôt que dérouler vingt appels, insérer `browser()` sous une condition :

```
if (length(x) == 1) browser()
```

« **Arguments to `browser` are ignored** » — on ne peut pas lui passer de condition, il faut l'entourer d'un `if`.

Étape 6 — se repérer dans la pile. Au prompt, `where` — « **Show the call stack** ». Et `q` pour « **halt execution and jump to the top-level immediately** » si l'on a vu ce qu'on voulait.

Étape 7 — le piège des noms. Si la fonction a une variable locale nommée `c` ou `n`, la taper **exécute la commande du débogueur**. « **its value can be accessed by using `get`** » → `get("c")`.

Étape 8 — nettoyer. `undebug(mafonction)`, sinon **chaque appel ultérieur** rouvre le débogueur. Et retirer les `browser()` du code : ils sont **invisibles** dans un script qui tourne en batch — où ils bloquent indéfiniment.

Étape 9 — la protection durable. Si la fonction modifie un état global (paramètres graphiques, `sink`, répertoire), poser `on.exit()` dès le début : « **guaranteed to be executed when the function exits either directly or as the result of a warning** » (§8.3). Une erreur ne laissera pas la session dans un état bancal.

Comment reconnaître le type d'exercice

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« interrompre avec un message »	<code>stop()</code>
« signaler sans interrompre »	<code>warning()</code>
« mes avertissements arrivent trop tard »	<code>options(warn = 1)</code>
« traiter les avertissements comme des bugs »	<code>options(warn = 2)</code>
« il y avait 12 avertissements »	plus de dix → <code>warnings()</code>
« restaurer un réglage quoi qu'il arrive »	<code>on.exit()</code>
« faire quelque chose à chaque erreur »	<code>options(error = ...)</code>
« arrêter R au premier problème en batch »	<code>options(error = expression(q("yes")))</code>
« d'où vient cette erreur ? »	<code>traceback()</code>
« s'arrêter à un endroit précis »	<code>browser()</code> , éventuellement sous un <code>if</code>
« suivre une fonction pas à pas »	<code>debug()</code> — puis <code>undebug()</code>
« déboguer une générique »	instrumenter la méthode , pas la générique
« savoir combien de fois c'est appelé »	<code>trace()</code>
« voir la pile depuis le débogueur »	<code>where</code>
« ma variable s'appelle <code>c</code> »	<code>get("c")</code>
« déboguer du code C »	pas avec ces outils — <code>gdb</code>
« uniformiser la présentation du code »	<code>keep.source = FALSE</code> + <code>dump()</code>
« où mon programme passe-t-il son temps ? »	<code>Rprof()</code> + <code>summaryRprof()</code>
« quelle fonction optimiser ? »	celle dont le <code>self</code> est élevé
« mon profil accuse une fonction inattendue »	peut-être le ramasse-miettes

Comment résoudre ce type d'exercice

Protocole « diagnostiquer une erreur » — 4 étapes.

1. `traceback()` immédiatement après — la chaîne d'appels.
2. Si c'est un avertissement qui dégénère : `options(warn = 2)` , puis retour à l'étape 1.
3. `debug()` sur la fonction suspecte, ou `browser()` sous condition au bon endroit.
4. `undebug()` et retirer les `browser()` avant de refermer.

Protocole « rendre une fonction sûre » — 3 étapes.

1. **Valider tôt** : `stop()` **avant** toute modification d'état (le patron `withdraw` de la fiche 310).
2. **Poser `on.exit()`** dès qu'un réglage global est touché — « *guaranteed to be executed ... either directly or as the result of a warning* ».
3. Ne jamais laisser un `browser()` dans du code livré.

Protocole « optimiser » — 4 étapes.

1. **Mesurer d'abord** : `Rprof("f.out")` ... `Rprof(NULL)` , puis `summaryRprof()` .
2. Lire `self` , pas `total` — le `total` d'une fonction qui délègue ne dit rien.
3. Se méfier des **exécutions courtes** : le ramasse-miettes fausse l'imputation → comparer **avec et sans `gc()`** préalable.
4. Puis seulement, chercher **la vectorisation** (fiche 301), **l'opération « objet entier »** (fiche 308), ou du **code compilé** (fiche 320).

● Common mistakes

Erreur	Correction
Croire qu'un <code>warning</code> interrompt	seul <code>stop</code> interrompt
Chercher un avertissement là où il s'affiche	à <code>warn = 0</code> il est différé → <code>options(warn = 1)</code>
Ignorer « There were 12 warnings »	<code>warnings()</code> les montre
Restaurer un réglage en fin de corps	une erreur saute la ligne → <code>on.exit()</code>
Croire <code>on.exit</code> exécuté après un avertissement seulement	il l'est dans les deux cas
Croire l'expression d'erreur exécutée après <code>on.exit</code>	elle l'est avant
Passer un argument à <code>browser()</code>	« Arguments to <code>browser</code> are ignored »
<code>debug()</code> sur une fonction générique	son corps est <code>UseMethod</code> — déboguer la méthode
Oublier <code>undebug()</code>	chaque appel ultérieur rouvre le débogueur
Laisser un <code>browser()</code> dans un script batch	il bloque indéfiniment
Taper <code>c</code> pour voir une variable nommée <code>c</code>	c'est une commande → <code>get("c")</code>
Attendre du débogueur qu'il entre dans du C	« no access to the statements in that language » → <code>gdb</code>
<code>trace()</code> sur un nom non identifiant	il faut des guillemets : <code>trace("[<-")</code>
Optimiser sans mesurer	profiler d'abord
Lire <code>total</code> au lieu de <code>self</code>	<code>total</code> inclut les appelées
Profiler une exécution très courte	le ramasse-miettes fausse le résultat
Profiler longtemps au pas par défaut	« the output files can be very large »
Croire <code>dump()</code> fidèle au source	« comments in your functions will be lost »

Ultimate Review

Deux mécanismes. « Functions such as `stop` or `warning` can be called directly or options such as `"warn"` can be used to control the handling of problems. » — le code signale, l'utilisateur décide.

`stop` . « *halts the evaluation of the current expression, prints the message argument and returns execution to top-level* ».

`warning` et l'option `warn` . négatif → « *ignored* » · 0 (défaut) → « *stored and printed after the top-level function has completed* » · 1 → « *printed as they occur* » · ≥ 2 → « *turned into errors* ». À `warn = 0` : moins de dix → imprimés ; plus de dix → « *a message indicating how many warnings occurred* » ; dans les deux cas `last.warning` contient le vecteur et `warnings()` y donne accès.

`on.exit` . « *stores the value of the body so that it will be executed when the function exits ... allows the function to change some system parameters and to ensure that they are reset* » · « *guaranteed to be executed when the function exits either directly or as the result of a warning* » · « *an error in the evaluation of the `on.exit` code causes an immediate jump to top-level* » · un seul argument : l'expression à évaluer.

Trois options. `warn` · `warning.expression` — « *the normal printing of warnings is suppressed if this option is set* » · `error` — « *the normal printing of error messages ... precedes the evaluation of the expression* ». « *Expressions installed by `options("error")` are evaluated before calls to `on.exit` are carried out.* » Exemple : `options(error = expression(q("yes")))` — « *an error will cause R to shut down and the global environment will be saved* ».

Le débogueur. « *Most debugging takes place either through calls to `browser` or `debug`. Both ... rely on the same internal mechanism ... The evaluation environment for the command is the currently active environment.* »

Les cinq commandes. `RET` — instruction suivante en `debug` , continuer depuis le `browser` · `c` / `cont` — continuer · `n` — instruction suivante, « *works from the browser as well* » · `where` — « *show the call stack* » · `Q` — « *halt execution and jump to the top-level immediately* ». Variable locale homonyme → `get("c")` . « *The debugger provides access only to interpreted expressions* » — pour le C, `gdb` .

`browser()` . « *causes R to halt execution at that point* » · « *Arguments to `browser` are ignored* ».

`debug()` / `undebug()` . « *each time that function is evaluated the debugger is invoked ... before each statement is executed the statement is printed out* » · l'exemple du manuel instrumente `mean.default` , pas `mean` .

`trace()` / `untrace()` . « *every time that function is evaluated the call to it is printed out* » · « *The name does not need to be quoted but for some functions you will need to quote the name* » ·

l'exemple `trace("<-")` rend visible le mécanisme ``*tmp*`` : `trace: "<-"(*tmp*, 3, value = 4)` .

`traceback()` . « *when an error has caused a jump to top-level a special variable called `.Traceback` is placed into the base environment ... one entry for each function call that was active at the time the error occurred* ».

Mettre au propre. « *code entered by users has the source code stored internally ... loading code from a package (by default) discards the source code* » · `keep.source = FALSE` ou `removeSource()` · la recette : `source("myfuns.R", keep.source = FALSE)` puis `dump(ls(all.names = TRUE), file = "new.myfuns.R")` , lancé par `R --vanilla < tidy.R` · « *comments in your functions will be lost* » · recommandation : `<-` plutôt que `=` .

Profiler. `Rprof("f.out") ... Rprof(NULL)` , puis `summaryRprof()` ou `R CMD Rprof` · « *recording at fixed intervals (by default every 20 msecs) which line in which R function is being used* » · CPU sur Unix, temps écoulé sur Windows · Total seconds : time spent in function and callees. Self seconds : time spent in function alone. — optimiser ce qui a un `self` élevé · « *often produces surprising results* » · trois réserves : pénalité de performance, fichiers très volumineux, et le ramasse-miettes imputé à qui le déclenche — d'où l'intérêt de comparer avec et sans `gc()` · outils plus fins : `proftools` et `profr` , « *these allow call graphs to be studied* ».

Active Recall

« *The exception handling facilities in R are provided through two mechanisms. Functions such as `stop` or `warning` can be called directly or options such as `"warn"` can be used to control the handling of problems.* » (§8)

Le partage est net : le code **signale** (`stop` , `warning`), l'utilisateur **décide** de ce qui en est fait (`options`).

C'est ce qui permet à `options(warn = 2)` de rendre strict un script indulgent **sans en changer une ligne** — les mêmes `warning()` deviennent alors des erreurs, donc traçables par `traceback()` .

« *The behaviour of a call to `warning` depends on the value of the option `"warn"` .* » (§8.2)

warn	Comportement
négatif	« <i>warnings are ignored</i> »
0 (défaut)	« <i>stored and printed after the top-level function has completed</i> »
1	« <i>printed as they occur</i> »
≥ 2	« <i>turned into errors</i> »

Et le détail du défaut : « *If there are fewer than 10 warnings they are printed after the function has finished evaluating. If there are more than 10 then a message indicating how many warnings occurred is printed. In either case `Last.warning` contains the vector of messages, and `warnings` provides a way to access and print it.* »

C'est l'explication du fameux « There were 12 warnings (use `warnings()` to see them) ».

« *The effect of a call to `on.exit` is to store the value of the body so that it will be executed when the function exits. This allows the function to change some system parameters and to ensure that they are reset to appropriate values when the function is finished. The `on.exit` is guaranteed to be executed when the function exits either directly or as the result of a warning.* » (§8.3)

Une ligne en fin de corps n'est atteinte que si tout s'est bien passé. Sur l'idiome `oldpar` de la fiche 314 :

```
oldpar <- par(no.readonly = TRUE)
on.exit(par(oldpar))           # <- execute QUOI QU'IL ARRIVE
```

Sans lui, une erreur au milieu du tracé **laisserait les réglages de l'utilisateur modifiés.**

« *An error in the evaluation of the `on.exit` code causes an immediate jump to top-level* ».

(§8.4)

Option	Rôle
<code>warn</code>	« <i>Controls the printing of warnings</i> »
<code>warning.expression</code>	une expression évaluée à chaque avertissement — « <i>the normal printing of warnings is suppressed</i> »
<code>error</code>	une expression évaluée à chaque erreur — « <i>the normal printing ... precedes the evaluation</i> »

L'ordre : « *Expressions installed by `options("error")` are evaluated before calls to `on.exit` are carried out.* »

Cela compte : au moment où l'expression d'erreur s'exécute, **les nettoyages n'ont pas eu lieu** — l'état des cadres est **encore intact**, ce qui rend l'inspection possible.

L'exemple du manuel : `options(error = expression(q("yes")))` — « *an error will cause R to shut down and the global environment will be saved* ».

« *There are five special commands that R interprets differently.* » (§9)

Commande	Effet
<code>RET</code>	« <i>Go to the next statement if the function is being debugged. Continue execution if the browser was invoked.</i> »
<code>c</code> / <code>cont</code>	« <i>Continue the execution</i> »
<code>n</code>	« <i>Execute the next statement ... This works from the browser as well</i> »
<code>where</code>	« <i>Show the call stack</i> »
<code>Q</code>	« <i>Halt execution and jump to the top-level immediately</i> »

Le piège : « *If there is a local variable with the same name as one of the special commands ... its value can be accessed by using `get`.* » Une variable locale `c` exige `get("c")` — c'est l'exemple même du §9.1.

`browser()` « *causes R to halt execution at that point* » (§9.1) — **un point d'arrêt posé dans le code.** « *Arguments to `browser` are ignored* », d'où la nécessité d'un `if` pour le conditionner.

`debug(fun)` — « *Subsequently, each time that function is evaluated the debugger is invoked ... Before each statement is executed the statement is printed out and a special prompt provided.* » (§9.2) — **un pas-à-pas sur toute la fonction, à chaque appel, jusqu'à `undebug()`.**

« *Both of these functions rely on the same internal mechanism and both provide the user with a special prompt.* »

Dans l'exemple du manuel, c'est `debug(mean.default)` et non `debug(mean)` : `mean` est une **générique** dont le corps est `UseMethod("mean")` (fiche 311).

```
> trace("[<-")
> x <- 1:10
> x[3] <- 4
trace: "[<->(*tmp*, 3, value = 4)
```

Il rend visible le mécanisme ``*tmp*`` de la fiche 302 : `x[3] <- 4` **est réellement** l'appel ``[<-`(*tmp*, 3, value = 4)`. Ce que le manuel décrivait comme « *as if the following had been executed* » est ici **observé en direct**.

Les guillemets : « *The name does not need to be quoted but for some functions you will need to quote the name in order to avoid a syntax error.* » (§9.3) — `[<-` n'est pas un identifiant (fiche 300), donc `trace([<-)` serait une erreur de syntaxe.

La différence avec `debug` : `trace` n'interrompt pas — il affiche l'appel et laisse continuer.

« *When an error has caused a jump to top-level a special variable called `.Traceback` is placed into the base environment. `.Traceback` is a character vector with one entry for each function call that was active at the time the error occurred. An examination of `.Traceback` can be carried out by a call to `traceback`.* » (§9.4)

C'est le premier réflexe après une erreur inattendue : le message dit **quoi**, `traceback()` dit **où**.

Il est particulièrement utile pour les pannes **différées** rencontrées dans les fiches précédentes — une classe posée sans vérification (fiche 311), un `drop` oublié (fiche 302), un `predict()` mal nommé (fiche 313) — où **le message ne mentionne pas la cause**.

« *By default code entered by users has the source code stored internally ... Loading code from a package (by default) discards the source code, and the function listing is re-created from the parse tree.* » (Writing R Extensions §3.1)

La recette — un fichier `tidy.R` contenant :

```
source("myfuncs.R", keep.source = FALSE)
dump(ls(all.names = TRUE), file = "new.myfuncs.R")
```

lancé par `R --vanilla < tidy.R`. « *Then the file `new.myfuncs.R` will contain the functions in alphabetical order in the standard layout.* »

Ce qu'on perd : « *Warning : comments in your functions will be lost.* »

Chaque morceau a sa raison : `keep.source = FALSE` fait oublier le texte d'origine ; `ls(all.names = TRUE)` prend **aussi les noms à point** (fiche 300) ; `dump` réécrit **depuis l'arbre syntaxique** (fiche 316) ; `--vanilla` isole des fichiers de démarrage.

L'alternative ponctuelle : `myfun <- removeSource(myfun)`.

```
Rprof("boot.out")  
# ... code lent ...  
Rprof(NULL)
```

puis `summaryRprof()` ou `R CMD Rprof boot.out` .

« *Profiling works by recording at fixed intervals (by default every 20 msecs) which line in which R function is being used* » (§3.2) — **CPU** sur Unix, **temps écoulé** sur Windows.

Les deux colonnes : « *Total seconds : time spent in function and callees. Self seconds : time spent in function alone.* »

C'est `self` qu'il faut lire. Dans l'exemple du manuel, `boot` a un **total de 100 %** et un **self de 0,2 %** : elle ne fait rien elle-même, elle appelle. **Optimiser une fonction à `self` faible n'apporte rien.**

« *This often produces surprising results and can be used to identify bottlenecks or pieces of R code that could benefit from being replaced by compiled code.* »

Trois (§3.2) :

1. « *profiling does impose a small performance penalty* » ;
2. « *the output files can be very large if long runs are profiled at the default sampling interval* » ;
3. « *Profiling short runs can sometimes give misleading results. R from time to time performs garbage collection to reclaim unused memory, and this takes an appreciable amount of time which profiling will charge to whichever function happens to provoke it. It may be useful to compare profiling code immediately after a call to `gc()` with a profiling run without a preceding call to `gc`.* »

La troisième est la plus insidieuse : le ramasse-miettes est **imputé à la fonction qui l'a déclenché**, laquelle n'en est pas responsable. Sur une exécution courte, le profil peut donc désigner un innocent.

Le remède du manuel : comparer deux profils, l'un précédé d'un `gc()` , l'autre non.

Pour aller plus loin : `proftools` et `profr` , « *these allow **call graphs** to be studied* ».

« *The debugger provides access only to interpreted expressions. If a function calls a foreign language (such as C) then no access to the statements in that language is provided. Execution will halt on the next statement that is evaluated in R. A symbolic debugger such as `gdb` can be used to debug compiled code.* » (§9)

Les outils de ce chapitre s'arrêtent à la frontière du code compilé. Un `.Call` s'exécute d'un bloc : on voit ce qui entre et ce qui sort, jamais ce qui se passe dedans.

C'est l'un des coûts de l'interfaçage avec C (fiche 320), et l'une des raisons de **ne descendre en C qu'après avoir profilé** — le code compilé est **plus rapide et moins observable**.

Flashcards

Question	Réponse
Les deux mécanismes de gestion des exceptions ?	Les fonctions (<code>stop</code> , <code>warning</code>) et les options
Que fait <code>stop()</code> ?	Arrête, affiche, revient au sommet
De quoi dépend le comportement de <code>warning()</code> ?	De l'option <code>warn</code>
<code>warn</code> négatif ?	Les avertissements sont ignorés
<code>warn = 0</code> ?	Stockés , imprimés après la fonction de haut niveau
<code>warn = 1</code> ?	Imprimés au moment où ils surviennent
<code>warn >= 2</code> ?	Transformés en erreurs
Quel est le défaut ?	0
Le seuil d'affichage à <code>warn = 0</code> ?	Dix
Au-delà, qu'affiche R ?	Un compte , pas les messages
Où sont-ils conservés ?	<code>last.warning</code>
Comment y accéder ?	<code>warnings()</code>
Que fait <code>on.exit()</code> ?	Il stocke une expression exécutée à la sortie
Quand est-elle garantie ?	Sortie directe ou par avertissement
Combien d'arguments prend-il ?	Un seul
Que se passe-t-il si le code d' <code>on.exit</code> échoue ?	Saut immédiat au sommet
Les trois options d'erreur ?	<code>warn</code> , <code>warning.expression</code> , <code>error</code>
Effet de <code>warning.expression</code> ?	L'affichage normal est supprimé
Et de <code>error</code> ?	L'affichage normal précède l'évaluation
Quel ordre entre <code>options("error")</code> et <code>on.exit</code> ?	L'expression d'erreur d'abord
Comment quitter R à la première erreur ?	<code>options(error = expression(q("yes")))</code>

Question	Réponse
Les deux principaux outils de débogage ?	<code>browser</code> et <code>debug</code>
Quel environnement au prompt ?	L'environnement actif
Combien de commandes spéciales ?	Cinq
Que fait <code>RET</code> en mode <code>debug</code> ?	Instruction suivante
Et depuis le browser ?	Il continue
Que fait <code>c</code> ?	Continuer
Que fait <code>n</code> ?	Instruction suivante — aussi depuis le browser
Que fait <code>where</code> ?	Affiche la pile d'appels
Que fait <code>q</code> ?	Arrêt immédiat , retour au sommet
Variable locale nommée <code>c</code> ?	<code>get("c")</code>
Le débogueur entre-t-il dans le code C ?	Non
Quel outil pour cela ?	<code>gdb</code>
Que fait <code>browser()</code> ?	Il arrête l'exécution à ce point
Accepte-t-il des arguments ?	Ils sont ignorés
Que fait <code>debug(f)</code> ?	Le débogueur s'ouvre à chaque appel
Comment l'arrêter ?	<code>undebug(f)</code>
Que faut-il déboguer sur une générique ?	La méthode , pas la générique
Que fait <code>trace(f)</code> ?	Il imprime l'appel à chaque évaluation
Interrompt-il ?	Non
Comment l'annuler ?	<code>untrace(f)</code>
Quand faut-il des guillemets ?	Quand le nom n'est pas un identifiant
Que montre <code>trace("[<-")</code> ?	Le mécanisme <code>`*tmp*`</code>

Question	Réponse
Qu'est-ce que <code>.Traceback</code> ?	Un vecteur de caractères des appels actifs
Où est-il placé ?	Dans l' environnement de base
Comment l'examiner ?	<code>traceback()</code>
Le code utilisateur garde-t-il sa source ?	Oui , par défaut
Et le code d'un paquet ?	Non — reconstruit depuis l' arbre syntaxique
Comment renoncer à la source ?	<code>keep.source = FALSE</code> ou <code>removeSource()</code>
Quelle fonction réécrit les objets ?	<code>dump()</code>
Dans quel ordre ?	Alphabétique , mise en page standard
Que perd-on ?	Les commentaires
Quel opérateur d'assignation est recommandé ?	<code><-</code> , plutôt que <code>=</code>
Quelle commande contrôle le profilage ?	<code>Rprof</code>
Comment l'arrêter ?	<code>Rprof(NULL)</code>
Le fichier par défaut ?	<code>Rprof.out</code>
L'intervalle par défaut ?	20 ms
Que mesure-t-on sous Unix ?	Le temps CPU
Et sous Windows ?	Le temps écoulé
Comment résumer ?	<code>summaryRprof()</code> ou <code>R CMD Rprof</code>
Que mesure la colonne <code>total</code> ?	La fonction et ses appelées
Et la colonne <code>self</code> ?	La fonction seule
Laquelle guide l'optimisation ?	<code>self</code>
Trois réserves sur le profilage ?	Pénalité, fichiers volumineux, ramasse-miettes
À qui le ramasse-miettes est-il imputé ?	À celui qui le déclenche

Question	Réponse
Comment s'en prémunir ?	Comparer avec et sans <code>gc()</code>
Deux paquets d'analyse plus fine ?	<code>proftools</code> et <code>profr</code>
Que permettent-ils ?	D'étudier les graphes d'appels